

Class Types in Non-Type Template Parameters

Document #: P0732R2
Date: 2018-06-06
Project: Programming Language C++
Audience: Evolution
Reply-to: Jeff Snyder <jeff-isocpp@caffeinated.me.uk>
Louis Dionne <ldionne.2@gmail.com>

1 TL;DR

We should allow non-union class types to appear in non-type template parameters. Require that types used as such, and all of their bases and non-static data members recursively, have a non-user-provided `operator<=>` returning a type that is implicitly convertible to `std::strong_equality`, and contain no references. Mangle values of those types by mangling each of their bases and non-static data members.

2 Introduction

Non-type template parameters are one of the few places in C++ where a visible distinction is made between class types and others: integral types, enum types, pointer types, point-to-member types, reference types and `std::nullptr_t` can all be used as non-type template parameters, but class types cannot. Array types can be used as non-type template parameters, but only syntactically—like function arguments of array type, they decay to pointers.

In C++98 there was a further distinction between class types and others: there were no compile-time constant expressions of class type. This changed in C++11 with the introduction of the `constexpr` keyword, leaving us in a situation where we have a great many tools to produce expressions of class type suitable for use as template arguments, but we cannot declare template parameters to accept them.

It would be desirable to remove this inconsistency in the language, but there are technical barriers to doing so. Jens Maurer’s 2012 paper[N3413] provides a detailed analysis of the problems involved. This paper proposes a way to resolve these problems, and start allowing the use of some class types in non-type template parameters.

3 Motivation

Beyond the more abstract benefits of generalising existing language features and removing seemingly-arbitrary inconsistencies/restrictions in the language, there are a few concrete use cases that are addressed by allowing the use of class types in non-type template parameters:

3.1 Wrapper Types

Many APIs define class types that wrap built-in types in order to improve type safety, turn nonsensical operations into compile-time errors, and customise the behaviour of various operators. A prominent example is the `std::chrono` library, where the `time_point` and `duration` types are typically represented as integers internally.

There are clear benefits of using abstractions such as those in `std::chrono` instead of integers holding (e.g.) numbers of nanoseconds, but they come at a hidden cost: a program that uses specific time or duration values in template arguments cannot take advantage `std::chrono`'s abstractions, because its types cannot be used in non-type template parameters.

3.2 Compile-Time String Processing

There have been multiple proposals[N3599][P0424R0] to relax the prohibition on string user-defined literals that take their raw form as individual `char` template arguments, and so far all of these proposals have failed to gain consensus.

The primary concern that gets raised in response to such proposals is that handling strings as packs of `char` template arguments is extremely costly in terms of the compiler's CPU and memory usage, and allowing such UDLs would make very large packs of `char` values much more common than they are today. This may in turn result in pressure on compiler vendors to make their products perform well under such compile-time string processing workloads, which is difficult due to the strings being represented using a syntax and mechanism that was never intended to support such use cases.

Recently, a proposal[P0424R2] to allow strings in template parameters has gained informal consensus, but the only valid arguments for those string template parameters are string literals and raw UDLs. Whilst this is significant progress, it still does not allow us to store computed strings in non-type template parameters.

By supporting non-type template parameters of class type, we can represent compile-time strings as objects of class type containing an array of characters (e.g. a `std::fixed_string`[P0259R0]), making the string content part of the value of the non-type template parameter. With this approach, any computed string can be stored in a non-type template parameter.

Why not just use a raw array as the non-type template parameter? Unfortunately, the syntactic space that this would occupy is already taken: since C++98, array-to-pointer decay happens in non-type template parameter declarations, thus it is already possible to declare a non-type template parameter of type `T[N]`, but what you actually get is a non-type template parameter of type `T*`.

Elsewhere in C++, the problems arising from array types not having value semantics are resolved by wrapping them in a class, c.f. `std::array`. Using the same approach here solves the problem of efficiently representing compile-time strings in non-type template parameters with a generic language feature, and does so in a manner consistent with the rest of C++.

4 Overview

At the core of the issue with allowing class types in non-type template parameters is the question whether of two template instantiations are the same, i.e. given `template<auto> int i;` and two values `a` and `b` of the same type, does the invariant¹ that `&i<a> == &i<b>` if and only if `a == b` holds? Current language rules ensure that it does, and maintaining this invariant seems desirable for the simplicity and teachability of the language.

Current language rules require both the compiler and the linker must be able to tell whether two template instantiations are the same or not, and so they must both be able to tell whether two values used as an argument to the same non-type template parameter are the same. This is typically achieved by including the values of all non-type template arguments in the mangled symbol name for the template instantiation, such that equality of the resulting string can be used as a proxy for equality of all of the template arguments.

If a template argument is a value of class type, then how do we mangle it into symbol names? If we cannot, then how does the linker tell whether two template arguments of class type are the same? By default, class types do not even have a notion of equality. If we disregard unions, we could say that classes are simply compared memberwise for the purposes of template instantiation. However, doing this would break the invariant that `&i<a> == &i<b>` if and only if `a == b`²: the latter may not even be valid, and if it is then it would only give the same results as the former if the relevant `operator==` implemented the same memberwise comparison that we used for the purposes of template instantiation.

To resolve this question, we must either:

1. Allow the existing invariant to be broken, and accept the complexity and subtleties added to the language by doing this, or

¹For brevity, this formulation of the invariant ignores reference types. The actual invariant is that `&i<a> == &i<b>` if and only if `REF_TO_PTR(a) == REF_TO_PTR(b)`, where `REF_TO_PTR(x)` is equivalent to `&x` if `decltype(x)` is a reference type, and `x` otherwise.

²See footnote 1

2. Attempt to develop the technology required for compiler and linkers to verify that user-defined equality operators are self-consistent and evaluate them, or
3. Only allow non-type template parameters of class type for classes which have memberwise equality

This paper proposes pursuing the last of these options, since it is the only option that maintains the aforementioned invariant without requiring substantial changes to current linker technology.

How do we tell whether a class type has memberwise equality? Detecting whether a user-defined `operator==` implements memberwise equality would require some heroic analysis by the compiler. If there were such a thing as compiler-generated comparisons, we could require that classes must have compiler-generated memberwise comparison instead of requiring that user-defined equality operators implement memberwise equality. C++ does not currently have any such compiler-generated comparisons, but Herb Sutter’s recent proposal[P0515R2] provides almost exactly what we need, by allowing `operator<=>` to be defaulted. To make use of this we need to modify the invariant to be “`&i<a> == &i<b>` if and only if `(a <=> b) == 0`”³, but this seems reasonable since `(a <=> b) == 0` is equivalent to `a == b` for all valid non-type template arguments in C++17.

By requiring that all classes used as non-type template parameters have an `operator<=>` that is defaulted within the class definition, and so do all of its members and bases recursively, we can ensure all of the following:

- The class type has a comparison operator available
- The class type has the same comparison operator available in all translation units
- The class’ comparison operator implements memberwise equality

Together, these guarantees allow us to determine whether two instantiations of a templated entity involving non-type template parameters of a type meeting the requirements above are the same, in a manner that is consistent with comparisons in user code, and without any substantial departure from current compiler and linker technologies.

5 Proposal

5.1 Conceptual Model

In current C++, the closest thing to having a non-type template parameter of class type is having separate non-type template parameters for each of its members (e.g. `template<int x_first, int x_second>` instead of `template<pair<int,int> x>`).

³The caveat from footnote 1 also applies here

This proposal uses that expansion as a conceptual model for classes in non-type template parameters—i.e. using a class type in a non-type template parameter is conceptually equivalent to having separate non-type template parameters for each of its bases, each of its non-array-type non-static data members, and each element of each of its array-type non-static data members.

This analogy is intended to aid in understanding the requirements for types and expressions used in non-type template parameters and arguments; it is not intended as an implementation model—there are likely to be more efficient ways of implementing this proposal, particularly with regard to arrays.

5.2 The Reference Problem

References have two notions of equality in current C++: `operator==` compares the referees, but instantiation of templates that have reference-type non-type template parameters compares the addresses of the referees.

This may become more counter-intuitive than it already is if there is a similar disparity in equality semantics for classes that have non-static data members of reference type. To avoid this problem, this paper does not propose allowing classes which contain references to be used in non-type template parameters.

5.3 The `operator==` Gap

This proposal aims to guarantee that whether two instances of a class are the same for the purposes of template instantiation is consistent with whether they are the same according to a comparison in normal C++ code.

Since the mechanism used to achieve this is based around `operator<=>`, any guarantees we can provide must also be in terms of `operator<=>`. For example, if we have a declaration `template<auto> int v`, then `&v<a> == &v<b>` is true iff `(a <=> b) == 0`.

Since there is nothing to prevent a class author from implementing an `operator==` that is inconsistent with `operator<=>`, we cannot guarantee that `&t<a> == &t<b>` is true iff `a == b`, however desirable this may be.

The practical consequence of this is that any generic code that relies on two template instantiations having the same address iff their arguments compare equal must compare the arguments using `operator<=>` directly, instead of `operator==`.

5.4 Calling member functions on template arguments

Being able to declare templates with non-type parameters of user-defined type and instantiate them is only half the picture. How do we then use these template parameters?

We would like to be able to call `const` member functions on these parameters at runtime, but for that to be possible they need to have an address.

To satisfy that requirement, we propose that template parameters of class type are `const` objects, of which there is one instance in the program for every distinct non-type template parameter value of class type.

5.5 Interaction with Class Template Argument Deduction

In keeping with the declaration and initialization of variables, we should support the use of class template argument deduction with non-type template parameters. Consider the following example, which initializes a non-type template parameter of type `std::fixed_string` (based on [P0259R0]) with the string "hello", but has to explicitly provide the string's length in order to do so:

```
namespace std {
    template <typename CharT, std::size_t N>
    struct basic_fixed_string
    {
        constexpr basic_fixed_string(const CharT (&foo)[N+1])
        { std::copy_n(foo, N+1, m_data); }
        auto operator<=>(const basic_fixed_string &,
                        const basic_fixed_string &) = default;
        CharT m_data[N+1];
    };

    template <typename CharT, std::size_t N>
    basic_fixed_string(const CharT (&str)[N])->basic_fixed_string<CharT, N-1>;

    template <std::size_t N>
    using fixed_string = basic_fixed_string<char, N>;
}

template <std::size_t N, std::fixed_string<N> Str>
struct A {};
```

```
using hello_A = A<5, "hello">;
```

By using template argument deduction for the `std::fixed_string` non-type template parameter, the declaration and use of `A` could be simplified to the following:

```
template <std::basic_fixed_string Str>
struct A {};
```

```
using hello_A = A<"hello">;
```

5.6 Usage with user-defined literals

To allow usage of the proposed functionality with user-defined literals, this paper proposes adopting the new form of user-defined literal operator templates for strings suggested in [P0424R2]:

```
template <std::basic_fixed_string str>
auto operator"" _udl();

"hello"_udl; // equivalent to operator""_udl<"hello">()
```

6 Wording

6.1 Remarks

This paper proposes extending the set of types that may appear in non-type template parameters with the addition of non-union literal class types that also have *strong structural equality*.

The intention of *strong structural equality* is to define the set of strongly comparable fundamental types, and class types for which all members and bases are strongly comparable, and comparison of the class type is equivalent to comparing all of its members and bases.

There is already a great deal of overlap between the set of types that can appear in non-type template parameters and the set of types that are both literal and have strong structural equality. This makes it possible to simplify [temp.param]p4 by making use of the term *strong structural equality*.

Introducing *strong structural equality* only handles the requirements on the types used in non-type template parameters. We must also consider the values that are permissible as arguments to non-type template parameters.

In keeping with our conceptual model, the restrictions on non-type template arguments of reference and pointer type in [temp.arg.nontype]p2 should apply to all reference- and pointer-type subobjects of class objects used as non-type template arguments.

For user-defined literals, we split the term of art *literal operator template* into two terms, *numeric literal operator template* and *string literal operator template*. The term *literal operator template* is retained and refers to either form. This is the approach that was originally taken by Richard Smith in [N3599].

We propose the feature test macro name `__cpp_nontype_template_parameter_class` for this feature.

6.2 Proposed Changes For Non-type Template Parameters

Change in [expr.prim.id.unqual] (8.4.4.1)p2

The result is the entity denoted by the identifier. If the entity is a local entity and naming it from outside of an unevaluated operand within the declarative region where the *unqualified-id* appears would result in some intervening *lambda-expression* capturing it by copy (8.4.5.2), the type of the expression is the type of a class member access expression (8.5.1.5) naming the non-static data member that would be declared for such a capture in the closure object of the innermost such intervening *lambda-expression*. [*Note*: If that *lambda-expression* is not declared `mutable`, the type of such an identifier will typically be `const` qualified. — *end note*] If the entity is a template parameter object for a template parameter of type `T`, the type of the expression is `const T`. Otherwise, the type of the expression is the type of the result. [*Note*: The type will be adjusted as described in 8.2.2 if it is cv-qualified or is a reference type. — *end note*] The expression is an lvalue if the entity is a function, variable, ~~or~~ data member, or template parameter object and a prvalue otherwise (8.2.1); it is a bit - field if the identifier designates a bit - field (11.5).

Change in [expr.const] (8.6)p2.7

- an lvalue-to-rvalue conversion (7.1) unless it is applied to
 - a non-volatile glvalue of integral or enumeration type that refers to a complete non-volatile const object with a preceding initialization, initialized with a constant expression, or
 - a non-volatile glvalue that refers to a subobject of a string literal (5.13.5), or
 - a non-volatile glvalue that refers to a non-volatile object defined with `constexpr` or a template parameter object (17.1 [temp.param]), or that refers to a non-mutable subobject of such an object, or
 - a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of `e`;

Change in [decl.type.class.deduct] (10.1.7.2)p4

For an expression `e`, the type denoted by `decltype(e)` is defined as follows:

- if `e` is an unparenthesized *id-expression* naming a structured binding (11.5), `decltype(e)` is the referenced type as given in the specification of the structured binding declaration;
- otherwise, if `e` is an unparenthesized *id-expression* naming a non-type template-parameter, `decltype(e)` is the type of the template-parameter after performing any necessary type deduction (10.1.7.4, 10.1.7.5).

- otherwise, if `e` is an unparenthesized *id-expression* or an unparenthesized class member access (8.2.5), `decltype(e)` is the type of the entity named by `e`. If there is no such entity, or if `e` names a set of overloaded functions, the program is ill-formed;
- otherwise, if `e` is an xvalue, `decltype(e)` is `T&&`, where `T` is the type of `e`;
- otherwise, if `e` is an lvalue, `decltype(e)` is `T&`, where `T` is the type of `e`;
- otherwise, `decltype(e)` is the type of `e`.

Change in `[decl.type.class.deduct]` (10.1.7.5)p2

A placeholder for a deduced class type can also be used in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (8.5.2.4), ~~or~~ as the *simple-type-specifier* in an explicit type conversion (functional notation) (8.5.1.3), or as the *type-specifier* in the *parameter-declaration* of a *template-parameter*. A placeholder for a deduced class type shall not appear in any other context.

Add the following paragraph at the end of `[class.compare.default]` (15.9.1):

A three-way comparison operator for a class type `C` is a structural comparison operator if it is defined as defaulted in the definition of `C`, and all three-way comparison operators it invokes are structural comparison operators.

A type `T` has *strong structural equality* if, for a glvalue `x` of type `const T`, `x <=> x` is a valid expression of type `std::strong_ordering` or `std::strong_equality` and either does not invoke a three-way comparison operator or invokes a structural comparison operator.

Change paragraph `[temp.param]` (17.1)p4 as follows:

A non-type *template-parameter* shall have one of the following (optionally cv-qualified) types:

- ~~integral or enumeration type,~~
- ~~pointer to object or pointer to function,~~
- a type that is literal, has strong structural equality (15.9.1), has no `mutable` or `volatile` subobjects, and in which if there is a defaulted member operator`<=>`, then it is declared public,
- an lvalue reference `type` to object or lvalue reference to function,
- ~~`std::nullptr_t`, or~~
- a type that contains a placeholder type (10.1.7.4), or
- a placeholder for a deduced class type (10.1.7.5).

Change paragraph [temp.param] (17.1)p6 as follows:

~~A non-type non-reference template-parameter is a prvalue. It shall not be assigned to or in any other way have its value changed. A non-type non-reference template-parameter cannot have its address taken.~~ When a non-type ~~non-reference~~ template-parameter of non-reference and non-class type is used as an initializer for a reference, a temporary is always used. An *id-expression* naming a non-type template-parameter of class type T denotes a static storage duration object of type `const T`, known as a *template parameter object*, whose value is that of the corresponding template argument after it has been converted to the type of the *template-parameter*. All such template parameters in the program of the same type with the same value denote the same template parameter object. [*Note:* If an *id-expression* names a non-type non-reference ~~template-parameter~~, then it is a prvalue if it has non-class type. Otherwise, if it is of class type T, it is an lvalue and has type `const T` (8.4.4.1). — *end note*]

[*Example:*

```
struct A { auto operator<=>(A, A) = default; };
template<const X& x, int i, A a> void f() {
    i++;                // error: change of template-parameter value
    &x;                  // OK
    &i;                  // error: address of non-reference template-parameter
    &a;                  // OK
    int& ri = i;         // error: non-const reference bound to temporary
    const int& cri = i;   // OK: const reference bound to temporary
    const A& ra = a;      // OK: const reference bound to a
                        // template parameter object
}
```

— *end example*]

Change paragraph [temp.arg.nontype] (17.3.2)p1 as follows:

If the type T of a *template-parameter* contains a placeholder type ~~or a placeholder for a deduced class type~~ (10.1.7.4, 10.1.7.5, 17.1), the ~~deduced parameter type is determined from the type of the template-argument by placeholder type deduction (10.1.7.4.1).~~ type of the parameter is the type deduced for the variable x in the invented declaration:

```
T x = template-argument ;
```

If a deduced parameter type is not permitted for a *template-parameter* declaration (17.1), the program is ill-formed.

Change paragraph [temp.arg.nontype] (17.3.2)p2 as follows:

A *template-argument* for a non-type *template-parameter* shall be a converted constant expression (8.6) of the type of the *template-parameter*. For a non-type *template-parameter* of reference or pointer type, or for each non-static data member of reference or pointer type in a non-type *template-parameter* of class type or subobject thereof, the reference or pointer value ~~of the constant expression~~ shall not refer to ~~(or for a pointer type, shall not~~ be the address of (respectively):

- a subobject (6.6.2),
- a temporary object (15.2),
- a string literal (5.13.5),
- the result of a `typeid` expression (8.5.1.8), or
- a predefined `__func__` variable (11.4.1).

Change the note at **[temp.type] (17.5)p4** as follows:

[*Note*: A string literal (5.13.5) is not an acceptable *template-argument* for a *template-parameter* of non-class type. [*Example*:

```
template<class T, T p> class X {
    /* ... */
};
```

```
X<const char*, "Studebaker"> x1; // error: string literal as template-argument
```

```
const char p[] = "Vivisectionist";
X<const char*,p> x2;                // OK
```

```
class A { constexpr A(const char*) {} };
```

```
X<A, "Pyrophoricity"> x3;           // OK: string literal is a constructor argument to A
```

— *end example*] — *end note*]

Change paragraph **[temp.type] (17.5)p1** as follows:

Two *template-ids* refer to the same class, function, or variable if

- their *template-names*, *operator-function-ids*, or *literal-operator-ids* refer to the same template and
- their corresponding type *template-arguments* are the same type and
- ~~their corresponding non-type template arguments of integral or enumeration type have identical values and~~
- ~~their corresponding non-type template arguments of pointer type refer to the same object or function or are both the null pointer value and~~

- their corresponding non-type *template-arguments* of pointer-to-member type refer to the same class member or are both the null member pointer value and
- their corresponding non-type *template-arguments* of reference type refer to the same object or function and
- their remaining corresponding non-type *template-arguments* have the same type and value after conversion to the type of the *template-parameter*, where they are considered to have the same value if they compare equal with `operator<=>` and
- their corresponding template *template-arguments* refer to the same template.

6.3 Proposed Changes for User-defined Literals

Replace “literal operator template” with “numeric literal operator template” in [lex.ext] (5.13.8)p3 and [lex.ext] (5.13.8)p4:

[...] Otherwise, *S* shall contain a raw literal operator or a numeric literal operator template (16.5.8) but not both. [...] Otherwise (*S* contains a numeric literal operator template), *L* is treated as a call of the form [...]

Change in [lex.ext] (5.13.8)p5:

If *L* is a *user-defined-string-literal*, let *str* be the literal without its *ud-suffix* and let *len* be the number of code units in *str* (i.e., its length excluding the terminating null character). If *S* contains a literal operator template with a non-type template parameter for which *str* is a well-formed *template-argument*, the literal *L* is treated as a call of the form `operator "" X<str>()`. Otherwise, the ~~The~~ literal *L* is treated as a call of the form `operator"" X(str, len)`.

Change in [over.literal] (16.5.8)p5:

~~The declaration of a literal operator template shall have an empty parameter-declaration-clause and its template-parameter-list shall have~~A numeric literal operator template is a literal operator template whose template-parameter-list has a single *template-parameter* that is a non-type template parameter pack (17.6.3) with element type `char`. A string literal operator template is a literal operator template whose template-parameter-list comprises a single non-type *template-parameter* of class type. The declaration of a literal operator template shall have an empty parameter-declaration-clause and shall declare either a numeric literal operator template or a string literal operator template.

7 Acknowledgements

Many thanks to Timur Doumler, Graham Haynes and Richard Smith for their comments on early drafts of this paper. Many thanks to Thomas Köppe and Richard Smith for their help with the core language wording for this feature.

References

- [N3413] Jens Maurer. Allowing arbitrary literal types for non-type template parameters. Proposal N3413, ISO/IEC JTC1/SC22/WG21, September 2012.
- [N3599] Richard Smith. Literal operator templates for strings. Proposal N3599, ISO/IEC JTC1/SC22/WG21, March 2013.
- [P0424R0] Louis Dionne. Reconsidering literal operator templates for strings. Proposal P0424R0, ISO/IEC JTC1/SC22/WG21, August 2015.
- [P0424R2] Louis Dionne. Reconsidering literal operator templates for strings. Proposal P0424R2, ISO/IEC JTC1/SC22/WG21, November 2017.
- [P0259R0] Michael Price & Andrew Tomazos. `fixed_string`: a compile-time string. Proposal P0259R1, ISO/IEC JTC1/SC22/WG21, February 2016.
- [P0515R2] Herb Sutter. Consistent comparison. Proposal P0515R1, ISO/IEC JTC1/SC22/WG21, September 2017.